

JAVA PROGRAMMING MASTERCLASS

介面 (Interface)

「定義行為契約，讓不同類別共享相同的能力」

[← 返回目錄](#)

Outline

- 認識介面 — 概念、語法、與抽象類別的比較
- 介面的成員變數 — `public static final`
- Java 8 新增介面內容 — Default 方法、Static 方法、Functional Interface
- Java 9 新增介面內容 — Private 方法
- 介面的繼承 — 基本繼承、多重繼承
- 進階主題 — 名稱衝突、Diamond 問題、密封介面 (Sealed)
- 課堂練習

認識介面

Interface

什麼是介面？

- 介面 (Interface) 不是物件，而是行為的規範
- 繼承描述「IS-A」關係，介面描述「有某種能力」
 - 鳥 (Bird) 是動物 → 繼承自 Animal (IS-A)
 - 鳥會飛、飛機也會飛 → 「飛」是**行為**，不是類別關係
 - 設計介面定義「飛」的行為，讓不同類別去實作 (implements)
- 介面無法用 `new` 直接建立物件

💡 比較飛機與鳥：兩者不是同一類時，但都有「飛」的能力 → 用介面定義共同行為

介面的語法

```
1 interface Flyable {  
2     void fly();    // 預設為 public abstract，可省略宣告  
3 }
```

- 所有方法預設是 **public abstract** (可省略)
- 所有成員變數預設是 **public static final**
- Java 8 之前介面只能有抽象方法

```
1 class Bird implements Flyable {  
2     @Override  
3     public void fly() {  
4         System.out.println("鳥兒飛翔");  
5     }  
6 }
```

介面的實作規則

- 實作類別必須覆寫所有的抽象方法
- 覆寫時存取控制必須宣告為 `public`
- 建議加上 `@Override` 確認覆寫正確

```
1  class Airplane implements Flyable {
2      @Override
3      public void fly() {
4          System.out.println("飛機用引擎飛");
5      }
6  }
7  public class Demo {
8      public static void main(String[] args) {
9          Flyable bird = new Bird();
10         bird.fly();
11     }
12 }
```


介面 vs 抽象類別

比較項目	抽象類別	介面
父類別	只能繼承一個	可繼承多個介面
子類別	只能 extends 一個	可 implements 多個
方法	可包含具體方法	Java 8 前只能抽象方法
用途	類別間的緊密關係	不同類別間的共同行為
適用	Car → Benz、Audi (IS-A)	Bird 與 Airplane 都能飛 (行為)

介面的成員變數

Interface Member Variables

介面的成員變數

介面的成員變數預設是 `public` 、 `static` 、 `final` ：

修飾詞	意義
<code>public</code>	所有人都可以取得
<code>static</code>	所有實作類別共享同一份
<code>final</code>	值不可更動（常數），一定要給預設值

```
1 interface Shape {  
2     double PI = 3.14159; // 等同 public static final  
3     double area();      // 等同 public abstract  
4 }
```

介面成員變數 – 範例

```
1  interface Shape {  
2      double PI = 3.14159;  
3      double area();  
4  }  
5  class Rectangle implements Shape {  
6      double width, height;  
7      Rectangle(double w, double h) { width = w; height = h; }  
8      public double area() { return width * height; }  
9  }  
10 class Circle implements Shape {  
11     double radius;  
12     Circle(double r) { radius = r; }  
13     public double area() { return PI * radius * radius; }  
14 }
```


Java 8 新增介面內容

Default & Static Methods

Java 8 新增介面內容 – 概覽

功能	Java 版本	說明
Constant variable	Java 7+	常數成員變數
Abstract methods	Java 7+	抽象方法
Default methods	Java 8 新增	預設方法 (有方法本體)
Static methods	Java 8 新增	靜態方法

💡 為什麼新增 **Default** 方法？為維持向後相容性，舊介面可以新增方法，不破壞已有的實作類別

Default 方法

```
1  interface Vehicle {  
2      String getBrand();  
3      default void alarmOn() {  
4          System.out.println("防盜啟動");  
5      }  
6      default void alarmOff() {  
7          System.out.println("防盜關閉");  
8      }  
9  }
```

- 使用 **default** 關鍵字，介面內直接提供預設實作
- 實作類別可直接繼承 Default 方法，也可以 Override
- Default 方法可以繼承，不強制是 abstract

Default 方法 – 範例

```
1  class Car implements Vehicle {
2      private String brand;
3      Car(String b) { this.brand = b; }
4      public String getBrand() { return brand; }
5      // alarmOn / alarmOff 直接繼承自 Vehicle，不需重寫
6  }
7  public class Demo {
8      public static void main(String[] args) {
9          Car car = new Car("Toyota");
10         car.alarmOn();    // 防盜啟動
11         car.alarmOff();   // 防盜關閉
12     }
13 }
```

Static 方法

```
1  interface MathUtils {  
2      static int add(int a, int b) {  
3          return a + b;  
4      }  
5      static int multiply(int a, int b) {  
6          return a * b;  
7      }  
8  }
```

- 在介面中定義 **static** 方法
- 只能用介面名稱呼叫，不能透過物件呼叫
- 實作類別無法 **Override** static 方法

Static 方法 – 範例

```
1 public class Demo {  
2     public static void main(String[] args) {  
3         int sum = MathUtils.add(3, 5);  
4         int product = MathUtils.multiply(4, 6);  
5         System.out.println("加法：" + sum);  
6         System.out.println("乘法：" + product);  
7     }  
8 }
```

💡 記住：Static 方法只能用「**介面名稱.方法名稱()**」呼叫，無法用物件呼叫

功能介面 (Functional Interface)

特性	說明
定義	只有一個抽象方法的介面 (又稱 SAM - Single Abstract Method)
註解	<code>@FunctionalInterface</code> (選用，但建議加上以利檢查)

```
1  @FunctionalInterface
2  interface Calculator {
3      int calculate(int a, int b);
4      // 可以有 default 或 static 方法，但只能有一個 abstract 方法
5  }
```

💡 主要用途：作為 Lambda 運算式的目標類型，簡化匿名內部類別的撰寫。

內建功能介面 (一) – 基本型別

java.util.function 套件提供常用的功能介面，可直接搭配 Lambda 使用：

介面	抽象方法	說明
Predicate<T>	boolean test(T t)	傳入一個值，回傳 true/false 判斷
Function<T,R>	R apply(T t)	傳入 T，轉換為 R 回傳
Consumer<T>	void accept(T t)	傳入一個值，執行動作不回傳
Supplier<T>	T get()	不傳入參數，產生一個 T

💡 這四個介面皆標有 **@FunctionalInterface**，可直接用 Lambda 或方法參考 (Method Reference) 傳入

內建功能介面 (一) – 範例

```
1  Predicate<String> isLong = s → s.length() > 5;
2  System.out.println(isLong.test("炭治郎"));           // false
3  System.out.println(isLong.test("Kamado Tanjiro"));    // true
4
5  Function<String, Integer> toLen = String::length;
6  System.out.println(toLen.apply("鬼滅之刃"));          // 4
7
8  Consumer<String> print = System.out::println;
9  print.accept("禰豆子"); // 禰豆子
10
11 Supplier<String> hero = () → "炭治郎";
12 System.out.println(hero.get()); // 炭治郎
```


內建功能介面 (二) – Bi 系列與 Operator

介面	抽象方法	說明
BiFunction<T,U,R>	R apply(T t, U u)	兩個不同型別輸入，一個輸出
UnaryOperator<T>	T apply(T t)	繼承 Function，輸入輸出型別相同
BinaryOperator<T>	T apply(T t1, T t2)	繼承 BiFunction，兩輸入同型別

```
1 BiFunction<String, Integer, String> repeat = (s, n) → s.repeat(n);
2 System.out.println(repeat.apply("鬼", 3)); // 鬼鬼鬼
3
4 UnaryOperator<String> upper = String::toUpperCase;
5 System.out.println(upper.apply("tanjiro")); // TANJIRO
```

Java 9 新增介面內容

Private Methods

Java 9 新增介面內容 – 概覽

功能	Java 版本	說明
Constant variable	Java 7+	常數成員變數
Abstract methods	Java 7+	抽象方法
Default methods	Java 8+	預設方法
Static methods	Java 8+	靜態方法
Private methods	Java 9 新增	私有方法
Private Static methods	Java 9 新增	私有靜態方法

Private 方法的規則

規則	說明
只能在介面內使用	無法從實作類別或外部呼叫
不能抽象化	必須有方法本體
<code>private static</code> 方法	可在 static 和 non-static 方法中使用
<code>private</code> non-static 方法	不能在 <code>private static</code> 方法中使用

💡 設計目的：讓介面內部程式碼可重複使用，避免在多個 Default 方法中寫重複邏輯

Private 方法 – 範例

```
1  interface LearnJava {  
2      default void method2() { method4(); }  
3      static void method3() { method5(); }  
4      private void method4() {  
5          System.out.println("這是private方法");  
6      }  
7      private static void method5() {  
8          System.out.println("這是private static方法");  
9      }  
10 }
```

介面的繼承

Interface Inheritance

繼承的三種關係

對象	關係	語法	限制
Class → Class	繼承	extends	只能一個父類別
Class → Interface	實作	implements	可多個，逗號分隔
Interface → Interface	繼承	extends	可多個父介面

- 當子介面繼承父介面，實作子介面的類別必須**同時實作所有**子介面與父介面的抽象方法

基本介面的繼承

```
1 interface Animal { void showMe(); }  
2 interface Bird extends Animal { void flying(); }
```

- **Bird** 繼承 **Animal**，實作 **Bird** 時需同時實作兩個方法

```
1 class Eagle implements Bird {  
2     public void showMe() { System.out.println("我是老鷹"); }  
3     public void flying() { System.out.println("老鷹展翅飛翔"); }  
4 }
```


介面多重繼承 – 概念

Java 類別不支援多重繼承，但介面支援：

```
1  interface B { void b(); }  
2  interface C { void c(); }  
3  class A implements B, C {  
4      public void b() { System.out.println("b方法"); }  
5      public void c() { System.out.println("c方法"); }  
6  }
```

- 一個類別可實作多個介面（逗號分隔）
- 一個介面可繼承多個介面

介面繼承多個介面 – 範例

```
1  interface Bird { void birdFly(); }
2  interface Airplane { void airplaneFly(); }
3  interface Fly extends Bird, Airplane {
4      void pediaFly();
5  }
6  class InfoFly implements Fly {
7      public void birdFly() { System.out.println("鳥翅飛翔"); }
8      public void airplaneFly() { System.out.println("引擎飛翔"); }
9      public void pediaFly() { System.out.println("百科飛翔"); }
10 }
```


類別實作多個介面

若多個介面有相同方法名稱，實作一次即可：

```
1  interface Bird { void flying(); }
2  interface Airplane { void flying(); }
3  class Fly implements Bird, Airplane {
4      @Override
5      public void flying() {
6          System.out.println("飛翔中");
7      }
8  }
```

💡 實作多個介面時，須重新定義所有父介面的抽象方法

繼承與實作規則摘要

對象	extends	implements
類別 (Class)	只能繼承一個父類別	可實作多個介面 (逗號分隔)
介面 (Interface)	可繼承多個父介面 (逗號分隔)	介面無法實作另一個介面
- 語法順序：<code>class A extends B implements C, D</code> - 介面無法 <code>new</code> 直接建立物件		

進階主題

Advanced Topics

實作時成員變數名稱衝突

不同介面有相同名稱的成員變數，使用介面名稱.成員變數存取：

```
1 interface B { int x = 5; }
2 interface C { int x = 8; }
3 class A implements B, C {
4     public void run() {
5         System.out.println(B.x); // 5
6         System.out.println(C.x); // 8
7     }
8 }
```

💡 因為介面成員變數是 **public static final**，可直接用介面名稱存取，不會產生衝突

類別重新定義 Default 方法

情況

解決方法

實作一個介面，想改變 Default 行為

直接 Override

實作多個有**相同** Default 方法名稱的介面

必須 **Override**，否則編譯錯誤

需要呼叫特定介面的 Default 方法

介面名稱.**super**.方法名稱()

類別重新定義 Default 方法 – 範例

```
1  interface Dog { default void running() { System.out.println("狗在跑");  
} }  
2  interface Cat { default void running() { System.out.println("貓在跑");  
} }  
3  class Pet implements Dog, Cat {  
4      @Override  
5      public void running() {  
6          Dog.super.running();  
7          Cat.super.running();  
8      }  
9  }
```


類別同時繼承類別與實作介面

語法：`extends` 在前，`implements` 在後

```
1 // class 子類別 extends 父類別 implements 介面名稱
2 class Pet extends Horse implements Dog {
3     @Override
4     public void running() {
5         System.out.println("寵物在跑");
6     }
7 }
```

💡 繼承類別（`extends`）必須排在實作介面（`implements`）之前，順序不能對調

繼承父類別與實作介面方法名稱衝突

當父類別與實作的介面有相同方法名稱，父類別方法的優先順序較高：

```
1  interface Dog { default void running() { System.out.println("介面跑");  
  } }  
2  class Horse { public void running() { System.out.println("馬在跑"); } }  
3  class Pet extends Horse implements Dog { }  
4  // Pet.running() → 執行 Horse 的 running()，父類別方法優先
```

💡 優先順序：子類別自訂 > 繼承的父類別方法 > 介面 Default 方法

多層次繼承中 Default 方法名稱相同

父子介面都有同名 Default 方法時，執行子介面的版本：

```
1  interface Animal {  
2      default void running() { System.out.println("動物跑"); }  
3  }  
4  interface Dog extends Animal {  
5      default void running() { System.out.println("狗在跑"); }  
6  }  
7  class Pet implements Dog { }  
8  // Pet.running() → 執行 Dog.running() (子介面覆蓋父介面)
```


鑽石 (Diamond) 問題

- 介面 B 與介面 C 同時繼承介面 A
- 三個介面都有同名的 Default 方法
- 類別 D 同時實作介面 B 與介面 C
- → 編譯器無法決定執行哪個版本，類別 D 必須強制 **Override**

💡 因繼承圖形的箭頭流向類似鑽石形狀，稱為鑽石問題 (**Diamond Problem**)

鑽石問題 – 程式範例

```
1  interface A { default void running() { System.out.println("跑A"); } }
2  interface B extends A { default void running() { System.out.println
("跑B"); } }
3  interface C extends A { default void running() { System.out.println
("跑C"); } }
4  class D implements B, C {
5      @Override
6      public void running() {
7          B.super.running(); // 跑B
8          C.super.running(); // 跑C
9      }
10 }
```

密封介面 (Sealed Interfaces)

特性	說明
關鍵字	<code>sealed</code>
控制權	使用 <code>permits</code> 指定哪些類別或介面可以實作它

```
1 public sealed interface Shape permits Circle, Rectangle {  
2     double area();  
3 }
```

💡 **JDK 17 正式特性：** 讓開發者能精確限制介面的擴充權限，增強系統安全性。

密封介面的實作規則

實作 **sealed** 介面的子類別必須明確宣告以下三種狀態之一：

狀態	說明
final	禁止再被繼承
sealed	繼續密封，並指定自己的子類別
non-sealed	解除密封，回歸傳統的開放繼承

```
1 public final class Circle implements Shape {  
2     public double area() { return 0; }  
3 }  
4 public non-sealed class Rectangle implements Shape {  
5     public double area() { return 0; }  
6 }
```

Records 實作介面 (Java 16)

Records 是隱式 **final**，天然符合 Sealed Interface **permits** 的要求，且自動生成 constructor 與 accessor：

```
1  sealed interface Shape permits Circle, Rectangle {  
2      double area();  
3  }  
4  record Circle(double radius) implements Shape {  
5      public double area() { return Math.PI * radius * radius; }  
6  }  
7  record Rectangle(double w, double h) implements Shape {  
8      public double area() { return w * h; }  
9  }
```

💡 Record 比 **final class** 更簡潔：不需手動寫 constructor、getter、equals()、hashCode()、toString()

密封介面與 Switch 窮舉性

當介面是 **sealed** 時，**switch** 可以檢查是否涵蓋所有可能的情况：

```
1  public double getArea(Shape shape) {  
2      return switch (shape) {  
3          case Circle c    → c.area();  
4          case Rectangle r → r.area();  
5          // 不需要 default，因為 Shape 是 sealed 且已窮舉所有子類別  
6      };  
7  }
```

💡 優勢：漏掉實作類別時，編譯器會報錯，比傳統 **instanceof** 更安全。

課堂練習

Practice

練習：設計介面階層

任務說明

設計一套介面與類別，模擬「跑」的行為：

1. 建立介面 **Runnable**，定義行為方法 **run()**
2. 類別 **Human**、**Car** 都實作 **Runnable**，各自定義不同的跑法
3. **Person** 繼承 **Human**，並重新定義 **run()** 方法
4. 執行後，將三者 (Human、Person、Car) 的跑法各自列印出來

練習：設計介面階層

解題提示

1. 定義介面 `Runnable`，加入抽象方法 `run()`
2. `Human` 與 `Car` 分別 `implements Runnable`，各自 Override `run()`
3. `Person extends Human`，Override `run()` 加上自己的行為
4. `main()` 中建立三個物件，分別呼叫各自的 `run()`

```
1  interface Runnable {  
2      void run();  
3  }  
4  class Human implements Runnable {  
5      public void run() { System.out.println("人在路上跑"); }  
6  }
```


Q & A

課程結束

介面定義行為契約，多重繼承讓設計更靈活